# Building a scalable Data Platform with snowflake, iceberg & S3 Tables

Jacob Verhoeks

Jacob.Verhoeks@gmail.com

https://www.linkedin.com/in/jacobverhoeks/

AWS community builder

SCHUBERG PHILIS

# “snowflake is expensive”

Everybody

# Let's integrate AWS & Snowflake

Jacob Verhoeks

Jacob.Verhoeks@gmail.com

https://www.linkedin.com/in/jacobverhoeks/

AWS community builder

SCHUBERG PHILIS

# S3 + Hive

/year=2025/month=09/day=18/<file1>.parquet

or Json csv

No index, files need to be read
No schema , inferred from the files
Append Only

# Competition

databricks

Delta lake on your s3

snowflake

Proprietary format on snowflake managed resources
Micro-partitions

# Iceberg

- Schema evolution
- Hidden partitions
- Scalability
- Time Travel
- ACID
  - Can handle updates & Deletes
  - Compliancy

# Iceberg Features

| Feature | v1 | V2 (current) | v3 |
|---|---|---|---|
| **Metadata File Size** | Grows indefinitely | **Bounded (fixed size + history pointers)** | Bounded (fixed size + history pointers) |
| **Snapshot History** | Inlined in each metadata file | **Split across metadata files (log entries)** | Split across metadata files (log entries) |
| **Delete File Support** | No native support | **Yes (Position and Equality Deletes)** | Yes (Position and Equality Deletes) |
| **Update/Delete Perf.** | Requires full file rewrites | **Highly efficient (write delete files + some data files)** | Highly efficient (write delete files + some data files) |
| **Sort Order Spec.** | Implicit (if any) | Implicit | **Explicit and Evolvable** |
| **Primary Goal** | Simple Table State | Scalable Metadata & Efficient Modifications | **Optimized Query Performance & Table Evolution** |
| **Upgrade Path** | Upgradeable to V2 | Upgradeable to V3 | Backward compatible with V2 |

# Writing to Iceberg

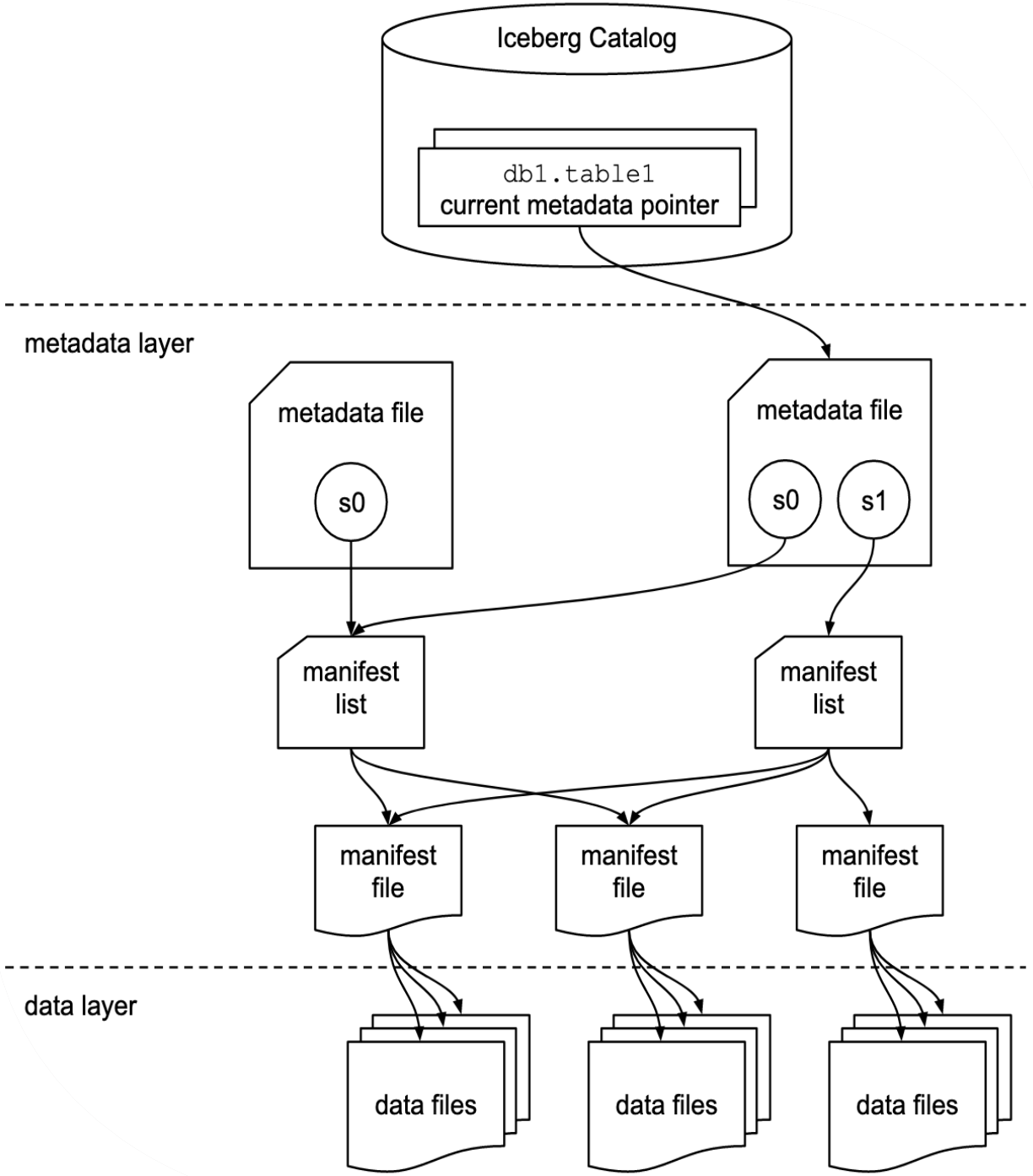
1. Writer prepares new data files
   Writes: s3://bucket/table/data/new-file-001.parquet

2. Writer creates new manifest
   Writes: s3://bucket/table/metadata/manifest-new.avro
   References: new-file-001.parquet + existing files

3. Writer creates new snapshot
   Writes: s3://bucket/table/metadata/snap-67890.avro
   References: manifest-new.avro

4. Writer updates table metadata (ATOMIC OPERATION)
   Writes: s3://bucket/table/metadata/v2.metadata.json
   Updates: current-snapshot-id = 67890

5. Commit complete - readers see new data atomically



# Metadata growth

Streaming Table Example (1 insert/minute):
- Day 1: 1,440 snapshots, 1,440 manifests
- Week 1: 10,080 snapshots, 10,080 manifests
- Month 1: 43,200 snapshots, 43,200 manifests
- Year 1: 525,600 snapshots, 525,600 manifests

Metadata Storage Growth:
- Snapshots: ~100KB each = 52GB/year
- Manifests: ~50KB each = 26GB/year

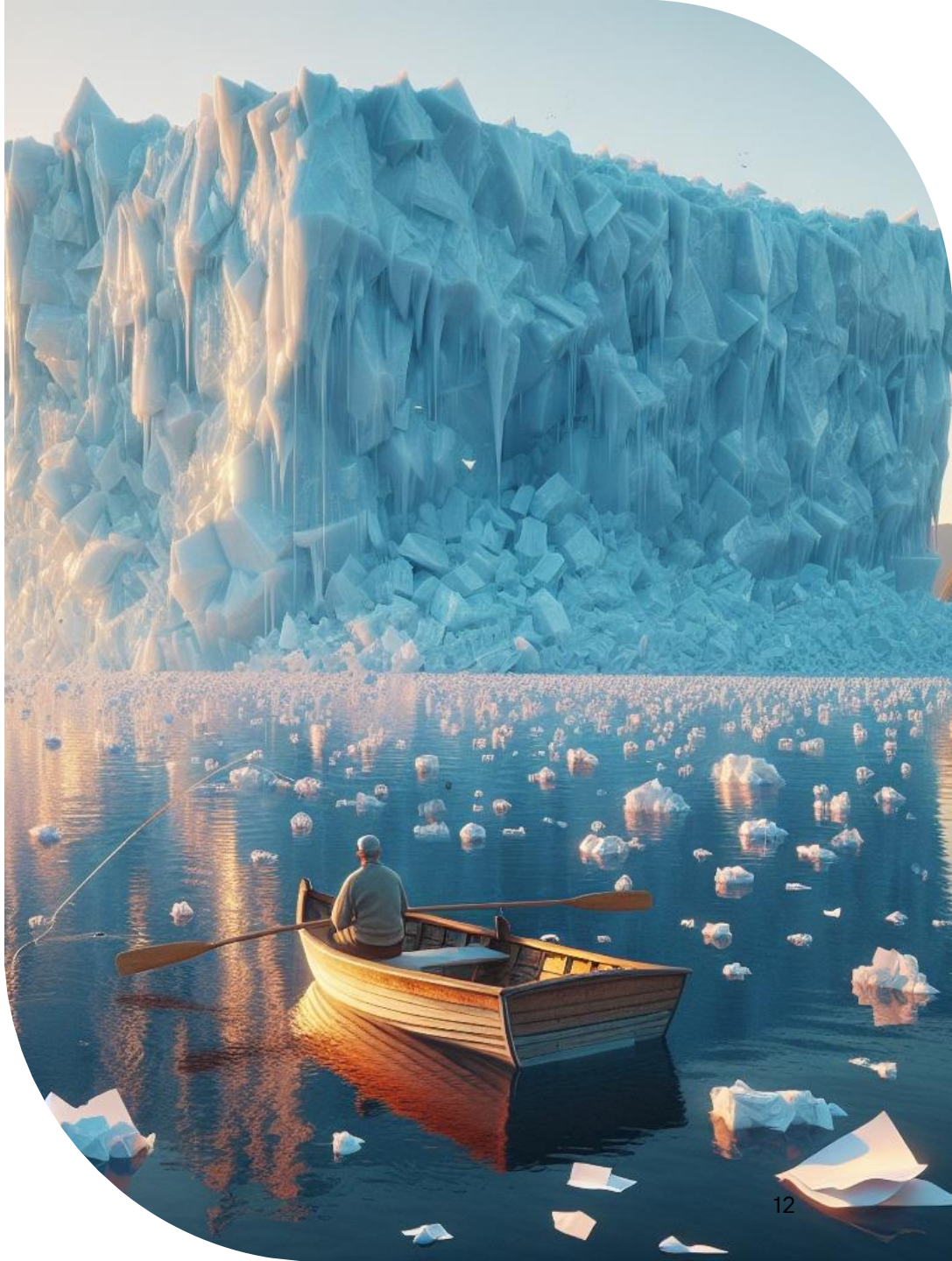
Total metadata: 78GB for 1 year of streaming!

# Query Planning

1. Read table metadata (1 S3 GET)
2. Read current snapshot (1 S3 GET)
3. Read manifest list (1 S3 GET)
4. Read ALL manifests (**N S3 GETs**)
5. Filter manifests by query predicates
6. Generate file list for scan

With 43,200 manifests (1 month streaming):

Query planning = **43,203 S3 API** calls before reading data!



# Maintenance

Metadata Compaction:
- Manifest file consolidation
- Snapshot expiration policies
- Orphan file cleanup
- Metadata size monitoring

Performance Optimization:
- File size optimization
- Partition layout tuning
- Sort order management
- Statistics collection

# S3 Tables

**S3 Tables** is S3 + Meta layer to manage Iceberg tables

Finegrained security using Lake Formation

No more direct access to S3
Built-in maintenance

Custom access methods, similar as S3 Vector

# Technical Catalogs

Central Management
Authentication
Authorization
Governance
Multi Engine

Examples:
AWS Glue Catalog
Databricks Unity

# Iceberg Rest API

Unite the iceberg world
Single Api for integration

Everything via HTTPS

| | | |
|---|---|---|
| GET | /v1/{prefix}/namespaces/{namespace}/tables/{table} | Load a table from the catalog |
| POST | /v1/{prefix}/namespaces/{namespace}/tables/{table} | Commit updates to a table |
| DELETE | /v1/{prefix}/namespaces/{namespace}/tables/{table} | Drop a table from the catalog |
| HEAD | /v1/{prefix}/namespaces/{namespace}/tables/{table} | Check if a table exists |
| GET | /v1/{prefix}/namespaces/{namespace}/tables/{table}/credentials | Load vended credentials for a table from the catalog |
| POST | /v1/{prefix}/tables/rename | Rename a table from its current name to a new name |
| POST | /v1/{prefix}/namespaces/{namespace}/tables/{table}/metrics | Send a metrics report to this endpoint to be processed by the backend |
| POST | /v1/{prefix}/transactions/commit | Commit updates to multiple tables in an atomic operation |

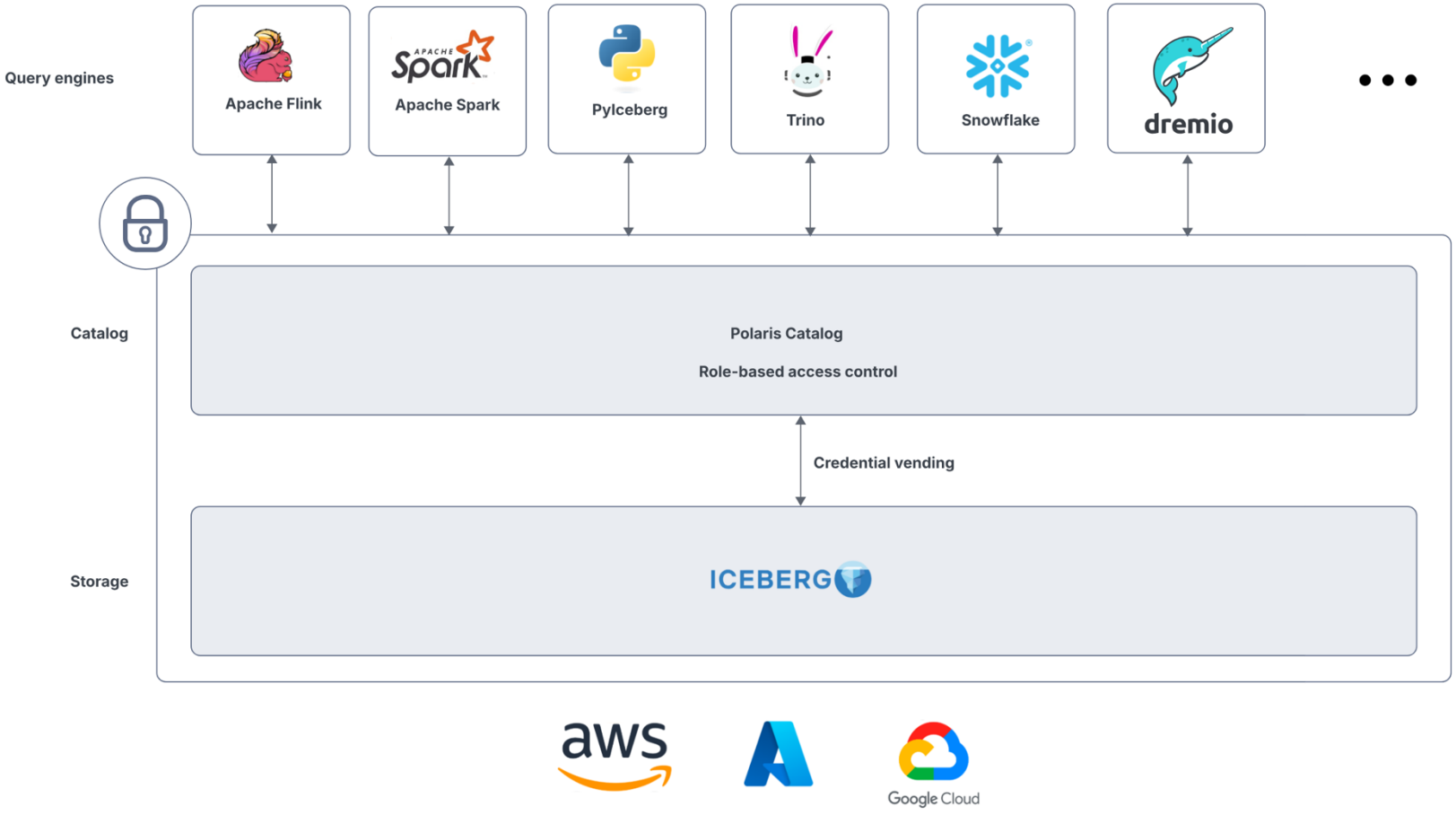
# Catalogs

| Type | Use-case | Pros | Cons |
|---|---|---|---|
| AWS Glue Catalog | AWS Native | **Serverless**<br>**IAM Integration**<br>**Cost-effective** | AWS Only<br>Limited Metadata |
| AWS Glue Iceberg Rest Catalog | Open | **Fine-grained Security**<br>**Rich metadata**<br>**S3Tables support** | More Complex<br>Lake formation |
| Apache Polaris | Open-source<br>Enterprise | **Open Source**<br>**Multi-Engine support**<br>**Advanced Governance** | Self-managed<br>Operational Overhead |
| Commercial (Dremio/Tabular) | Managed Service | **Fully Managed**<br>**Performance optimized**<br>**Enterprise features** | Vendor Lock-in<br>Cost |

# Apache Polaris

Started by Snowflake
Apache Incubator

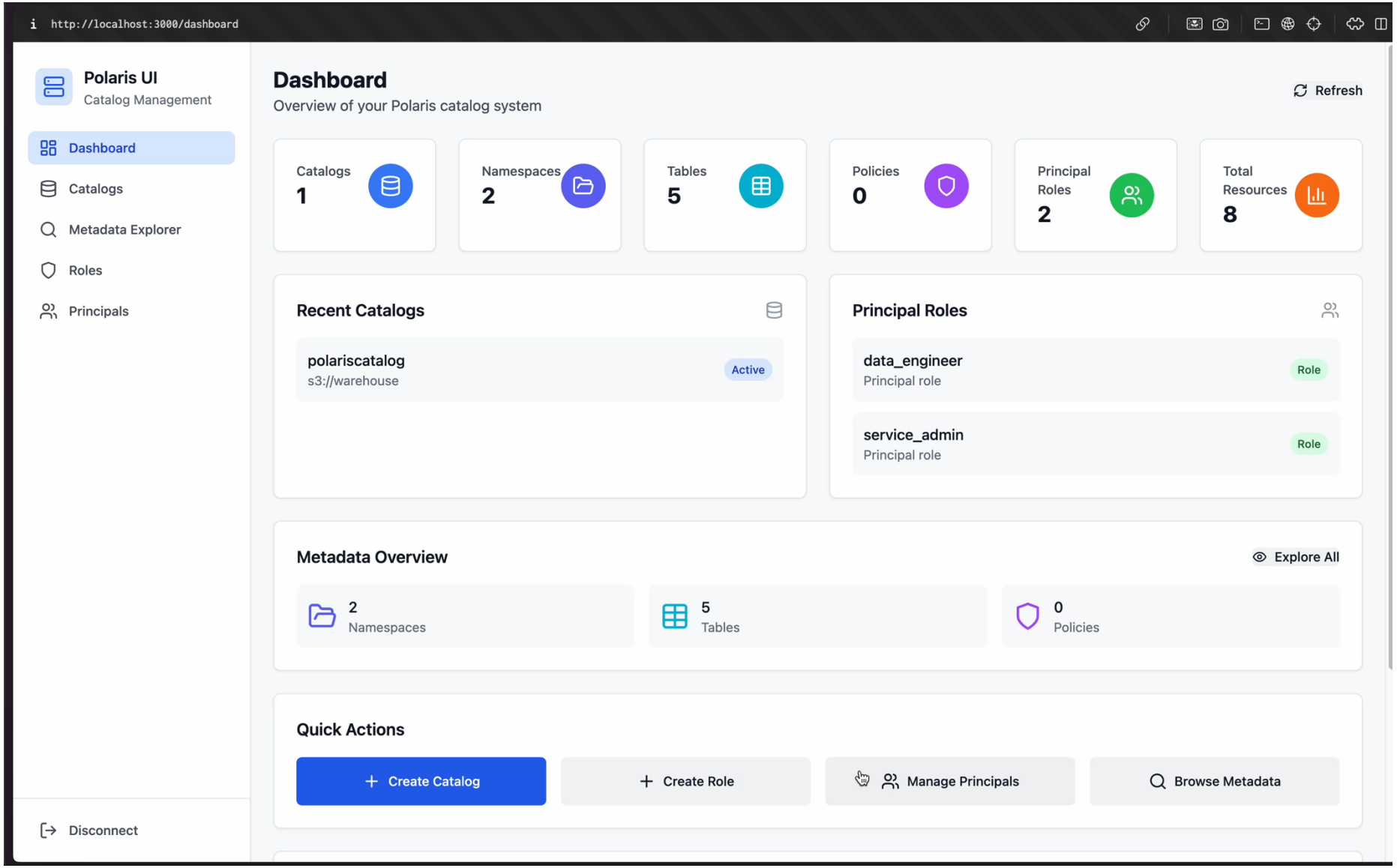
Rest-Api Only



# Polaris UI

Created with vscode + cline claude 4

By Rafael Herrero



# snowflake

# Snowflake expensive?

Saas , all – in – one
Simplified credit model everything to credits/hours

$2.60 standard $3.90 Enterprise $5.20 Business Critical

Compute warehouse:

| | | |
|---|---|---|
| X-Small | 1 / hour | 16 GB Memory |
| to | | |
| 6X-Large | 512 / hour | 1 TB Memory |

# Snowflake expensive?

Processing large amounts of data require:

Compute time +  memory

On snowflake can this can be costly

# Combine with AWS

Process on AWS and integrate with Snowflake

S3 + Hive integration

raw s3 access, interpret in Snowflake

S3 + Glue integration

Catalog role

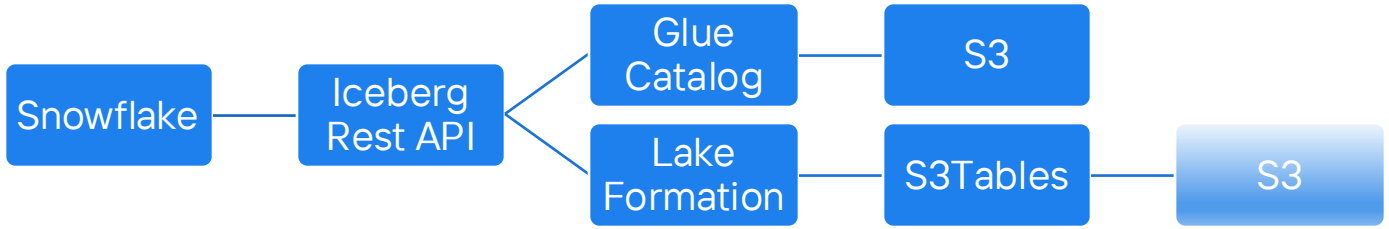
Storage role

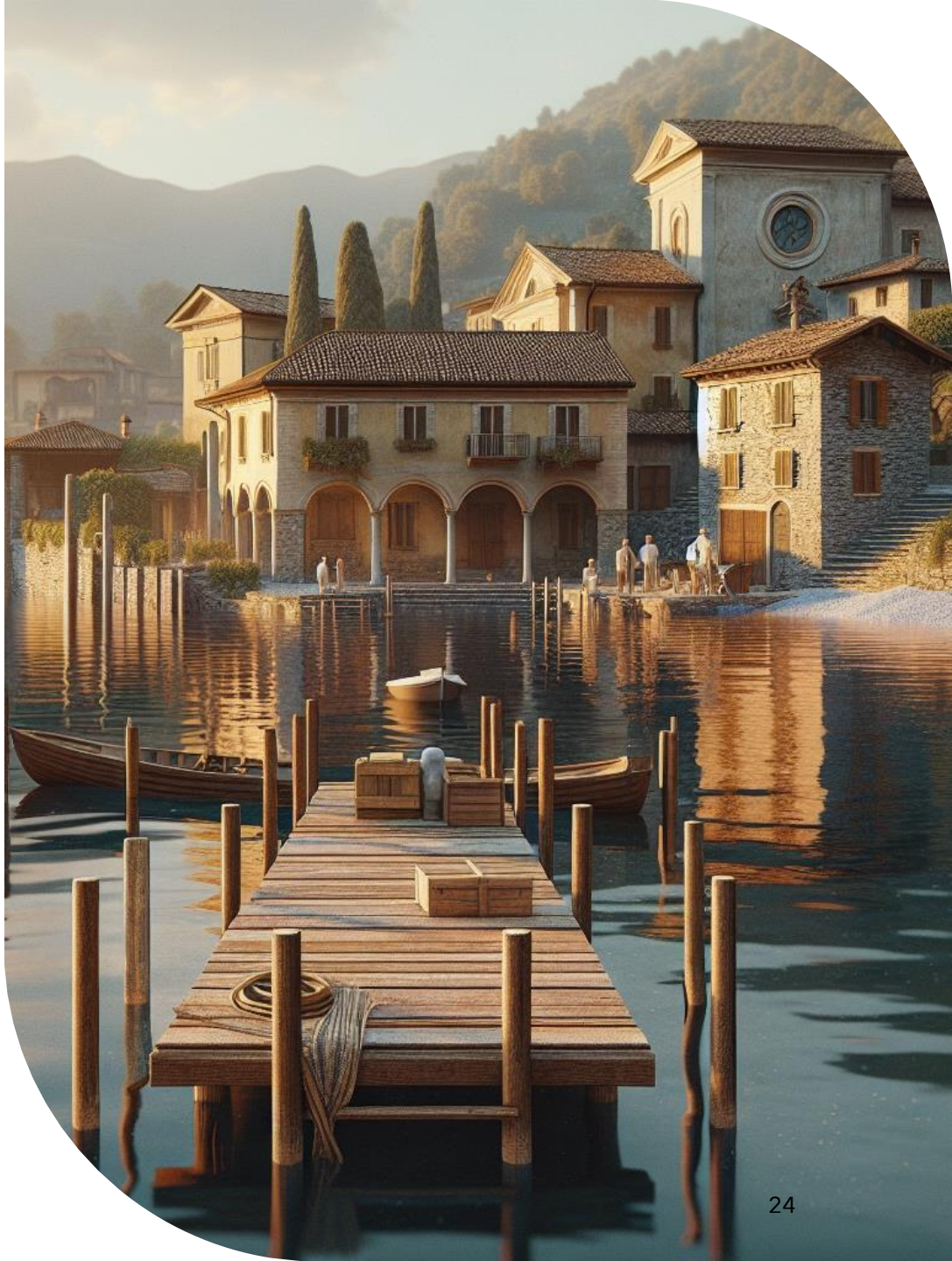
Sync

Glue Iceberg Rest API Integration

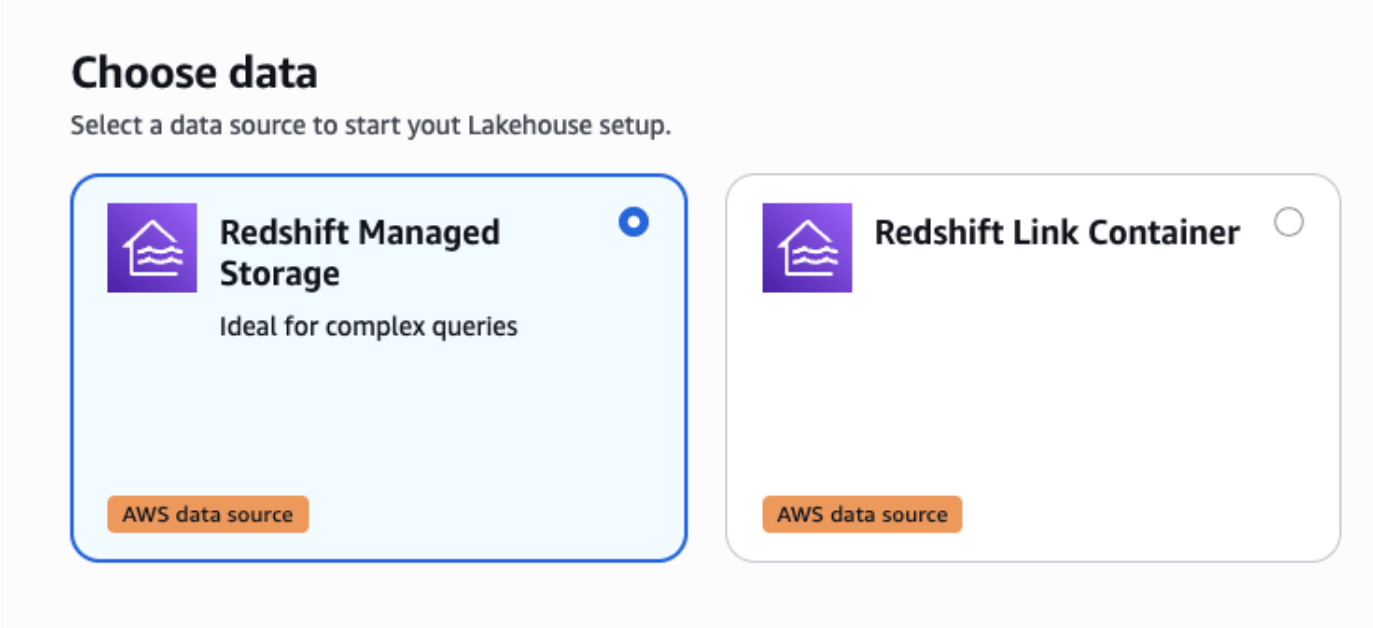
Perfect Solution?

# Architecture



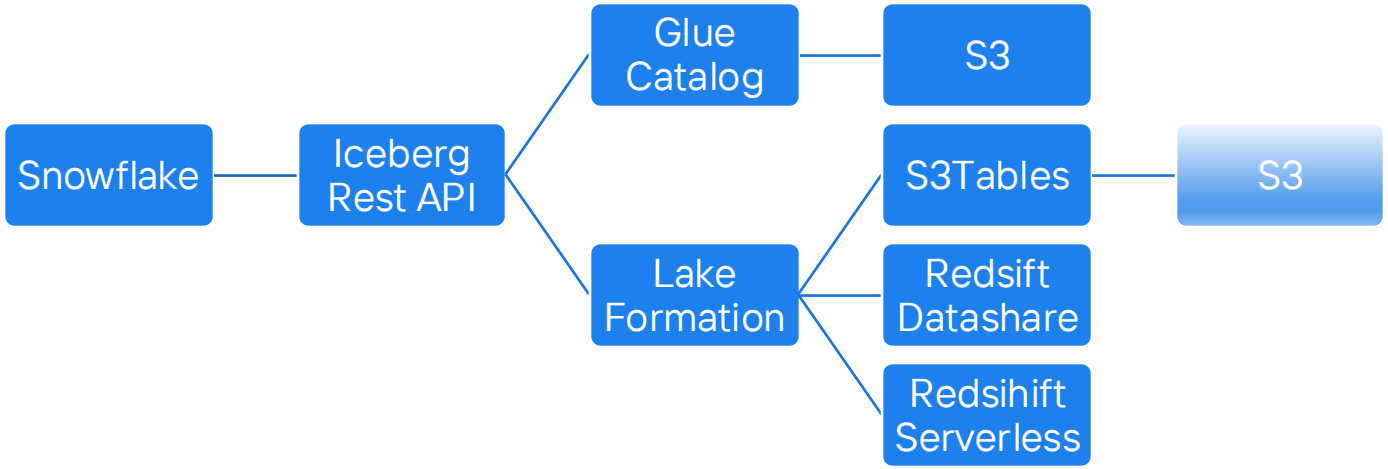
# SageMaker LakeHouse




Connecting factor in

AWS DataZone
AWS SageMaker Lakehouse

Provide uniform access to
S3 and RedShift



# Let’s connect a S3 bucket with iceberg tables

# S3 + Iceberg: Create Table

```sql
CREATE TABLE iceberg_test_table (
  id int,
  data string,
  category string)
PARTITIONED BY (category, bucket(16,id))
LOCATION 's3://iceberg-demo-iceberg-data/iceberg-test-table'
TBLPROPERTIES (
  'table_type'='ICEBERG',
  'format'='parquet',
  'write_compression'='snappy',
  'optimize_rewrite_delete_file_threshold'='10'
)
```

# S3 + Iceberg: View Table

Editor | Recent queries | Saved queries | Settings

Workgroup: primary

**Data**

Data source: AwsDataCatalog

Catalog: None

Database: iceberg_demo_analytics

**Tables and views** Create

Filter tables and views

Tables (2) 1

- iceberg_test_table
- iceberg_user_events

Views (0) 1

Query 3

```
1 SELECT * FROM "iceberg_demo_analytics"."iceberg_user_events" limit 10;
```

SQL Ln 1, Col 1

Run again | Explain | Cancel | Clear | Create

Reuse query results
up to 60 minutes ago

Query results | Query stats

Completed — Time in queue: 92 ms — Run time: 759 ms — Data scanned: 1.08 KB

**Results** (10)

Copy | Download results CSV

Search rows

| # | user_id | event_type | region | data |
|---|---|---|---|---|
| 1 | 4567891234 | purchase | ap-south | {"item_id":"sku123","amount":29.99} |
| 2 | 9012345678 | logout | ap-south | {"session_duration":7200} |
| 3 | 5678901234 | purchase | us-east | {"item_id":"sku789","amount":99.50} |

# S3 + Iceberg: Catalog Integration

```sql
-- create iceberg catalog pointing to database on glue
CREATE or REPLACE CATALOG INTEGRATION glue_iceberg_s3_catalog
  CATALOG_SOURCE = ICEBERG_REST
  TABLE_FORMAT = ICEBERG
  CATALOG_NAMESPACE = 'iceberg_demo_analytics'
  REST_CONFIG = (
    CATALOG_URI = 'https://glue.eu-central-1.amazonaws.com/iceberg'
    CATALOG_API_TYPE = AWS_GLUE
    CATALOG_NAME = '311141556126'
)
REST_AUTHENTICATION = (
    TYPE = SIGV4
    SIGV4_IAM_ROLE = 'arn:aws:iam::311141556126:role/iceberg-demo-snowflake-role'
    SIGV4_SIGNING_REGION = 'eu-central-1'
)
ENABLED = TRUE;

-- get the user and external id
DESCRIBE CATALOG INTEGRATION glue_iceberg_s3_catalog;
```

# S3 + Iceberg Credentials output

| | property | property_type | property_value | property_default |
|---|---|---|---|---|
| 1 | ENABLED | Boolean | true | false |
| 2 | API_AWS_IAM_USER_ARN | String | arn:aws:iam::888184096481:user/y1u51000-s | |
| 3 | API_AWS_EXTERNAL_ID | String | [redacted] | |
| 4 | CATALOG_SOURCE | String | ICEBERG_REST | |
| 5 | CATALOG_NAMESPACE | String | iceberg_demo_analytics | |
| 6 | TABLE_FORMAT | String | ICEBERG | |
| 7 | REFRESH_INTERVAL_SECONDS | Integer | 30 | 30 |
| 8 | REST_CONFIG | EnumMap | {CATALOG_URI=https://glue.eu-central-1.amazonaws.c | |
| 9 | REST_AUTHENTICATION | EnumMap | {TYPE=SIGV4, SIGV4_IAM_ROLE=arn:aws:iam::311141 | |
| 10 | COMMENT | String | | |

# S3 + Iceberg External Storage

```sql
-- create role for external volume access
CREATE OR REPLACE EXTERNAL VOLUME iceberg_s3_ext_vol
    STORAGE_LOCATIONS =
        (
            (
                NAME = 'iceberg_s3'
                STORAGE_PROVIDER = 'S3'
                STORAGE_BASE_URL = 's3://iceberg-demo-iceberg-data/'
                STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::311141556126:role/iceberg-demo-snowflake-storage-role'
                STORAGE_AWS_EXTERNAL_ID = '[redacted]'
            )
        )
        ALLOW_WRITES = FALSE;

-- get the user and external id
DESCRIBE EXTERNAL VOLUME iceberg_s3_ext_vol;
```

# S3 + Iceberg
# Create Table

```sql
-- test the external volume and catalog integration
SELECT SYSTEM$VERIFY_EXTERNAL_VOLUME('iceberg_s3_ext_vol');
SELECT SYSTEM$VERIFY_CATALOG_INTEGRATION('glue_iceberg_s3_catalog');

-- create iceberg table , map to the catalog and external volume
CREATE or replace ICEBERG TABLE iceberg_user_events
  CATALOG = glue_iceberg_s3_catalog
  EXTERNAL_VOLUME = iceberg_s3_ext_vol
  CATALOG_TABLE_NAME = 'iceberg_user_events'
  AUTO_REFRESH = TRUE;
```

# S3 + Iceberg Create Table


```
{
  "success": true,
  "storageLocationSelectionResult": "PASSED",
  "storageLocationName": "iceberg_s3",
  "servicePrincipalProperties": "STORAGE_AWS_IAM_USER_ARN: arn:aws:iam::888184096481:user/y1u51000-s; STORAGE_AWS_EXTERNAL_ID: NH80631_SFStorageRole=2_
  "location": "s3://iceberg-demo-iceberg-data/",
  "storageAccount": null,
  "region": "eu-central-1",
  "writeResult": "UNVERIFIED because the external volume doesn't allow writes",
  "readResult": "UNVERIFIED because the external volume doesn't allow writes",
  "listResult": "UNVERIFIED because the external volume doesn't allow writes",
  "deleteResult": "UNVERIFIED because the external volume doesn't allow writes",
  "awsRoleArnValidationResult": "PASSED",
  "azureGetUserDelegationKeyResult": "SKIPPED"
}
```

```
{ "success" : true, "errorCode" : "", "errorMessage" : "" }
```


| | USER_ID | EVENT_TYPE | TIME_STAMP | REGION | DATA |
|---|---|---|---|---|---|
| 1 | 9012345678 | logout | 2025-09-15 18:50:40.000 | ap-south | {"session_duration":7200} |
| 2 | 7890123456 | click | 2025-09-15 12:00:00.000 | us-west | {"element":"banner_ad","page":"land |
| 3 | 3456789012 | login | 2025-09-14 23:10:55.000 | ap-south | {"ip":"10.0.0.15","device":"desktop"} |
| 4 | 1234567890 | login | 2025-09-15 10:15:23.000 | us-west | {"ip":"192.168.1.10","device":"mobil |
| 5 | 5678901234 | purchase | 2025-09-15 16:20:30.000 | us-east | {"item_id":"sku789","amount":99.50} |

Query Details

Query duration 848ms

Rows 5

Query ID 01bf2e25-0205-35a9-0...

Show more

USER_ID

1234567890 9012345678

EVENT_TYPE

100% filled

Ask Copilot

# S3 + Iceberg

```sql
-- create role for external volume access
CREATE OR REPLACE EXTERNAL VOLUME iceberg_s3_ext_vol
    STORAGE_LOCATIONS =
        (
            (
                NAME = 'iceberg_s3'
                STORAGE_PROVIDER = 'S3'
                STORAGE_BASE_URL = 's3://iceberg-demo-iceberg-data/'
                STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::311141556126:role/iceberg-demo-snowflake-storage-role'
                STORAGE_AWS_EXTERNAL_ID = '[redacted]'
            )
        )
        ALLOW_WRITES = FALSE;

-- get the user and external id
DESCRIBE EXTERNAL VOLUME iceberg_s3_ext_vol;
```

# Let’s connect a S3Tables

# S3Tables: Lake formation

```
# S3 Bucket for Iceberg tables
resource "aws_s3tables_table_bucket" "iceberg_data" {
  name = "${var.project_name}-table-iceberg-data"

  lifecycle {
    prevent_destroy = false
  }
}


# S3 Tables namespace
resource "aws_s3tables_namespace" "demo" {
  table_bucket_arn = aws_s3tables_table_bucket.iceberg_data.arn
  namespace        = "table_demo_analytics"

  lifecycle {
    prevent_destroy = false
  }
}
```

```
resource "aws_s3tables_table" "user_events" {
  name             = "table_user_events"
  namespace        = aws_s3tables_namespace.demo.namespace
  table_bucket_arn = aws_s3tables_table_bucket.iceberg_data.arn
  format           = "ICEBERG"

  lifecycle {
    prevent_destroy = false
  }

  metadata {
    iceberg {
      schema {
        field {
          name     = "user_id"
          required = true
          type     = "long"
        }
        field {
          name     = "event_type"
          required = true
          type     = "string"
        }
        field {
          name     = "time_stamp"
          required = true
          type     = "timestamp"
        }
        field {
          name     = "data"
          required = true
          type     = "string"
        }
        field {
          name     = "region"
          required = true
          type     = "string"
        }
      }
    }
  }
}
```

# S3Tables: Lake formation

AWS Lake Formation

Dashboard

Data Catalog

Catalogs New

Databases

Tables

Views

Data filters

Data sharing

## Catalogs

### How it works

**Create a catalog**
Register Redshift databases as catalogs in the Data Catalog. Learn more

**Manage catalog permissions**
Manage permissions for specific catalogs, databases, tables and fine-grained data access. Learn more

**Access from query editors**
Access catalog objects from Redshift Query Editor v2 and Athena Console.

Automatically create a catalog from an S3 table bucket. Allows external engines to access data in Amazon S3 locations with full table access. Enable S3 Table integration

### Enable S3 Table integration

Once integration is enabled, every table bucket in this account and region will automatically be available under the s3tablescatalog catalog in AWS Data Catalog.

**Select a principal to register**
AWS LakeFormation needs to be able to call S3 Tables APIs on your behalf to retrieve S3 Table buckets, namespace, and tables. S3 tables must be registered with AWS LakeFormation with an IAM role that can be assumed.

iceberg-demo-lf-s3tables-role

☑ Allow external engines to access data in Amazon S3 locations with full table access

Cancel Enable

When you register an S3 tables catalog in Lake Formation, the registering principal automatically receives permissions to perform all supported Lake Formation operations on the registered data location.

Actions View Create catalog

A catalog is the top level in the Data Catalog's three-level data hierarchy and contains Data Catalog objects.

Find catalogs by name

| Name | Type | Source | Owner account ID | Shared resource | Shared resource o... | Shared resource o... |
|---|---|---|---|---|---|---|
| 311141556126 | Default | Default catalog | 311141556126 | - | - | - |
| s3tablescatalog | Federated | S3 | 311141556126 | - | - | - |

# S3Tables: Lake formation

## Application integration settings Learn more

### Application integration settings

Use the options below to control which third-party engines are allowed to read and filter data in Amazon S3 locations registered with Lake Formation.

- [ ] Allow external engines to filter data in Amazon S3 locations registered with Lake Formation

Check this box to allow third-party engines to access data in Amazon S3 locations that are registered with Lake Formation.

- [x] Allow external engines to access data in Amazon S3 locations with full table access

When you enable this option, Lake Formation will return credentials to the integrated application directly without IAM session tag validation.

Cancel Save

### LF-Tags or catalog resources

Choose a method to grant permissions.

- ( ) Resources matched by LF-Tags (recommended)
  Manage permissions indirectly for resources or data matched by a specific set of LF-Tags.
- (•) Named Data Catalog resources
  Manage permissions for specific databases or tables, in addition to fine-grained data access.

**Catalogs**

Choose catalogs

311141556126:s3tablescatalog/iceberg-demo-table-iceberg-data ×

**Databases**

Select one or more databases.

Choose databases

table_demo_analytics ×
311141556126:s3tablescatalog/iceberg-demo-table-iceberg-data

**Tables - *optional***

Select one or more tables.

Choose tables

**Views - *optional***

Select one or more views.

Choose views

**Data filters - *optional***

Select one or more data filters.

Choose data filters | Load more | Create new

Manage data filters

### Database permissions

**Database permissions**

Choose specific access permissions to grant.

- [ ] Create table
- [ ] Alter
- [ ] Drop
- [x] Describe
- [ ] Super

This permission is the union of all the individual permissions to the left, and supersedes them.

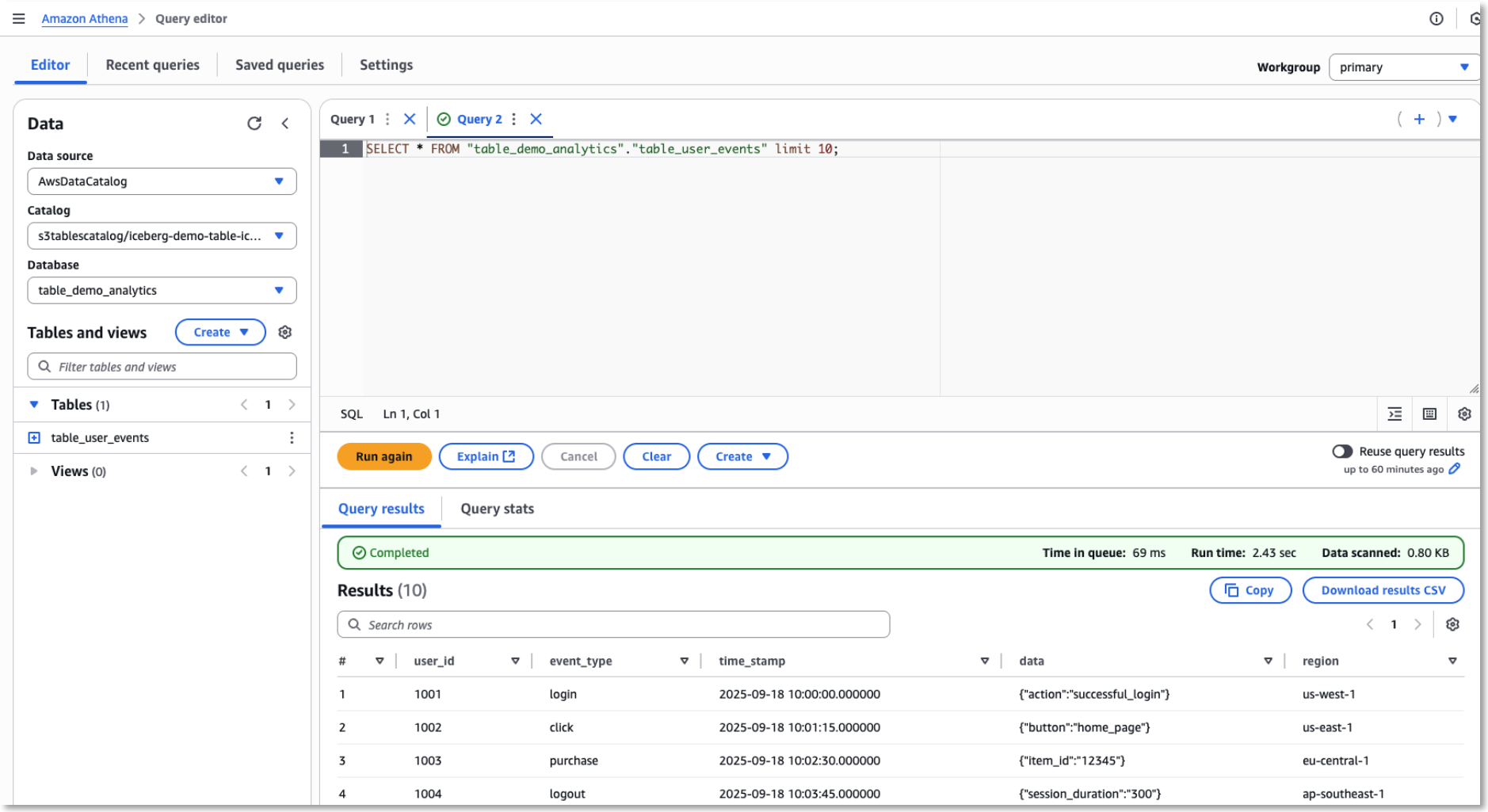
**Grantable permissions**

Choose the permission that can be granted to others.

- [ ] Create table
- [ ] Alter
- [ ] Drop
- [x] Describe
- [ ] Super

This permission allows the principal to grant any of the permissions to the left, and supersedes those grantable permissions.

# S3Tables: Lake formation



# S3Tables: Catalog Integration

```sql
-- create the S3 tables integration in lakeformation
-- enable external access in lakeformation
-- create the catalog integration for s3 tables, note catalog_namespace = glue database

CREATE OR REPLACE CATALOG INTEGRATION s3tablescatalog
  CATALOG_SOURCE = ICEBERG_REST
  TABLE_FORMAT = ICEBERG
  CATALOG_NAMESPACE = 'table_demo_analytics'
  REST_CONFIG = (
    CATALOG_URI = 'https://glue.eu-central-1.amazonaws.com/iceberg'
    CATALOG_API_TYPE = AWS_GLUE
    CATALOG_NAME = '311141556126:s3tablescatalog/iceberg-demo-table-iceberg-data'
    ACCESS_DELEGATION_MODE = VENDED_CREDENTIALS
  )
  REST_AUTHENTICATION = (
    TYPE = SIGV4
    SIGV4_IAM_ROLE = 'arn:aws:iam::311141556126:role/iceberg-demo-snowflake-role'
    SIGV4_SIGNING_REGION = 'eu-central-1'
  )
  ENABLED = TRUE;
```

# S3Tables
# Catalog Integration

```sql
-- get the user and external id
DESCRIBE CATALOG INTEGRATION s3tablescatalog;

-- test the connection
SELECT SYSTEM$VERIFY_CATALOG_INTEGRATION('s3tablescatalog');
```

| | property | property_type | property_value | property_default |
|---|---|---|---|---|
| 1 | ENABLED | Boolean | true | false |
| 2 | API_AWS_IAM_USER_ARN | String | arn:aws:iam::888184096481:user/y1 | |
| 3 | API_AWS_EXTERNAL_ID | String | [redacted] | |
| 4 | CATALOG_SOURCE | String | ICEBERG_REST | |
| 5 | CATALOG_NAMESPACE | String | table_demo_analytics | |
| 6 | TABLE_FORMAT | String | ICEBERG | |
| 7 | REFRESH_INTERVAL_SECONDS | Integer | 30 | 30 |
| 8 | REST_CONFIG | EnumMap | {CATALOG_URI=https://glue.eu-centra | |
| 9 | REST_AUTHENTICATION | EnumMap | {TYPE=SIGV4, SIGV4_IAM_ROLE=arn: | |
| 10 | COMMENT | String | | |

```
1  { "success" : true, "errorCode" : "", "errorMessage" : "" }
```

# S3Tables
# Catalog Integration

```sql
-- grant describe on the catalog and table lakeforamtion to get access
-- create iceberg table , map to the catalog, no creds needed as vended credentials are used
CREATE or replace ICEBERG TABLE table_user_events
  CATALOG = s3tablescatalog
  CATALOG_TABLE_NAME = 'table_user_events'
  AUTO_REFRESH = TRUE;
```

```sql
77  select * from demo_db.public.table_user_events;
```

Results | Chart

| | USER_ID | EVENT_TYPE | TIME_STAMP | DATA | REGION |
|---|---|---|---|---|---|
| 1 | 1001 | login | 2025-09-18 10:00:00.000 | {"action":"successful_login"} | us-west-1 |
| 2 | 1002 | click | 2025-09-18 10:01:15.000 | {"button":"home_page"} | us-east-1 |
| 3 | 1003 | purchase | 2025-09-18 10:02:30.000 | {"item_id":"12345"} | eu-central-1 |
| 4 | 1004 | logout | 2025-09-18 10:03:45.000 | {"session_duration":"300"} | ap-southeast-1 |
| 5 | 1005 | view | 2025-09-18 10:05:00.000 | {"page":"product_list"} | us-west-2 |

Query Details

Query duration 95ms

Rows 20

Query ID 01bf2e2a-0205-3410-0...

Show more

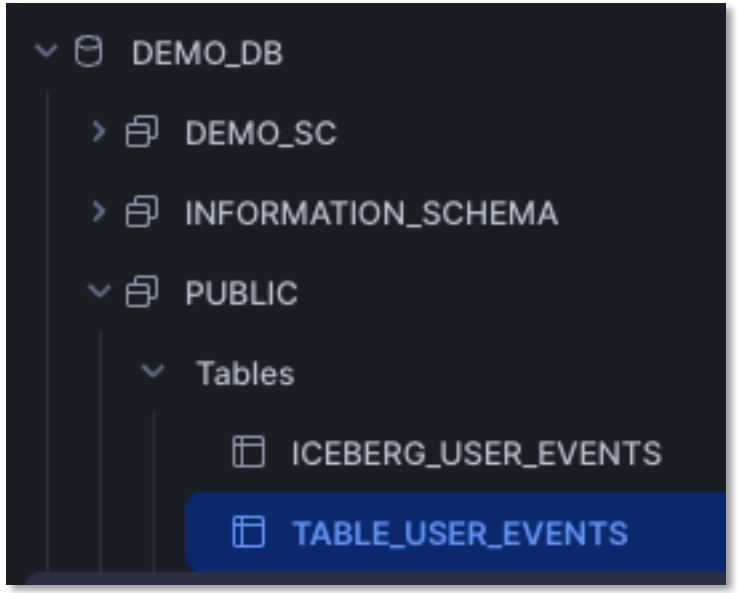
# Table Integration

- 2 tables from difference sources in the same schema
- No schema definition required

- Could be any account

- Seaming less for snowflake users

- We can do all the processing in AWS or other tools, while keeping the tight integration



SCHUBERG PHILIS

# How does it performs

# Performance

| Query | S3 Iceberg Athena | S3tables Athena | S3 via Snowflake (no-cache) | S3 tables vis Snowflake (no-cache) |
|---|---|---|---|---|
| Select * | 5.7s | 4.5s | 2.3ms (1.2s) | 80ms (5.3s) |
| Date, group by | 3.9s | 3.0s | 888ms (**400ms**) | 219ms (2s) |
| Json parse, group by | 4.0s | 4.0s | 2ms (554ms) | 2.6ms (2.5s) |

1M rows generated data

Snowflake caching works also connected iceberg tables

Test on your own data and volumes

# Performance

| Query | S3 Iceberg Via (raw) duckdb | S3tables via duckdb | Pyiceberg duckdb | S3 Iceberg athena | S3tables athena |
|---|---|---|---|---|---|
| Select * | 2.5s | 0.8s | 9.8s | 5.7s | 4.5s |
| Date, group by | 2.6s | 0.9s | | 3.9s | 3.0s |
| Json parse, group by | 2.7s | 0.9s | | 4.0s | 4.0s |

Duckdb Support for iceberg Rest API currently doesn't work with Glue v4 authentication

Pyiceberg has rest api support, but in beginning stage

SCHUBERG PHILIS

# Cheaper?

Don’t do it for the storage, but analyze your compute cost

# Storage Cost

- 10 TB / month

- AWS S3 $ 230
- AWS S3Tables $ 270 + premium on requests

- Snowflake $ 230 + 25% overhead= ~ $ 300

SCHUBERG PHILIS

# Conclusion

Iceberg wins

# Conclusion

- Iceberg Rest API opens the data lake house world
- AWS + Snowflake + Databricks + Dremio + others

- S3Tables offers iceberg data + management
- Price different is minimal, the big cost-factor is in the compute

- Snowflake easy integration for Read and Write
- Snowflake caching speeds up iceberg tables

- Open-source tools are slowly getting support
- AWS Sig V4 vs standard oauth2 support

# Thanks

## Jacob Verhoeks

Feedback


- Jacob Verhoeks
- Jacob.Verhoeks <dot> gmail.com
- https://www.linkedin.com/in/jacobverhoeks/